SAMSUNG INNOVATION INTERNSHIP 2027 | ROUTE A

Spaza Stock & Sales Tracker

A practical inventory and point-of-sale platform for small retailers.

Prepared by Romeo Mduduzi Mangena | 13 November 2026

01 Executive summary

The Spaza Stock & Sales Tracker brings product records, counter sales, stock visibility, and restocking decisions into one lightweight web application. Owners and staff can find products, record transactions, and review shop activity from a single interface.

This is a working full-stack prototype designed around small-retailer workflows. Business impact is an intended outcome, not a measured result.

02 The operational need

Manual stock and sales records can make it difficult to keep quantities current, notice low-stock items early, and estimate what to reorder. The solution supports consistent sale recording and a clearer view of inventory without requiring complex retail systems.

03 Core workflows

- Product catalogue: maintain names, categories, prices, quantities, suppliers, and reorder thresholds; search and filter the catalogue.
- Point of sale: record a product and quantity, calculate the sale total, record the staff username, and reduce stock in the same transaction. Requests exceeding available stock are rejected.
- Restocking: flag products at or below their reorder threshold. The suggested quantity targets max(2 x reorder threshold, 10 units), less current stock, with a minimum suggestion of one unit. Estimated cost uses the product price.
- Shop overview: review recorded revenue, sales count, product count, low-stock count, inventory value, and sales history.

INTENDED VALUE

More consistent records, earlier low-stock visibility, and better-informed restock planning.

Implementation and evaluation

A simple client-to-service architecture keeps shop workflows connected.

SHOP INTERFACE	APPLICATION API	DATA STORE
React 18	**Spring Boot 3**	**JPA database**
Dashboard, inventory, POS	Java 21, JWT, business rules	H2 locally; PostgreSQL on Render

04 Technology and deployment

The responsive React 18 and Tailwind CSS interface is bundled with Vite and served by a Java 21 / Spring Boot 3 application. REST APIs handle authentication, product management, sales, restock suggestions, and dashboard summaries. Spring Data JPA provides persistence. H2 supports local development and tests; the Render Blueprint provisions PostgreSQL for deployment.

05 Access and safeguards

- JWT-based authentication and BCrypt password encoding.
- Owner permissions include product create, update, delete, and restock; owner and staff roles can record sales.
- Transactional sale processing checks available stock before decrementing it and saving the sale.
- Production owner credentials and JWT configuration are supplied through environment variables; demo accounts are not seeded in production.

06 Evaluation and reviewer access

Automated verification: mvn -o test passed all 13 tests on 13 November 2026. A regression test confirms the production JavaScript and CSS load without authentication.

The sign-in page outage was caused by Spring Security returning 403 for frontend files under /assets/. The path is now permitted so the login interface can load; shop APIs still require authentication.

Public landing page (overview):
https://romeo725.github.io/Spaza-Stock-Sales-Tracker/src/main/resources/static/landing.html

Public app sign-in (Codespaces; available while active):
https://scaling-space-barnacle-r6qxxr67wqrfwqj-8080.app.github.dev/

Local app URL: http://localhost:8080. Start it with mvn spring-boot:run; localhost is reachable only from the computer running the app.

- Local owner demo: username owner; password password123.
- Local staff demo: username staff; password password123.

These demo credentials are for local evaluation only. GitHub Pages hosts the static landing page; it does not host the Spring API.

NEXT STEPS

Potential extensions: barcode scanning, supplier ordering workflows, and demand forecasting.

These are roadmap ideas, not features in the current implementation.